

Simulating the Ethereum market

Jelle Roessink, Vincent Scheinhardt

May 2026

1 Introduction

In this report we show how we simulated the Ethereum market mempool under EIP-1559 with a dynamic base fee. At $t = 0$, the mempool is empty and the base fee is $b_0 = 10$ Gwei. When and how this changes will be explained in this report.

2 Events

In our simulation, there are 4 events:

- TransactionArrival
- BlockProduction
- Expiry
- UpdateBaseFee

We also have a Transaction object, which stores its own gas demand, max fee and tip, and has a pointer to its own expiry event. These events are described in Figure 1. Our simulation terminates after a specific amount of simulated time.

2.1 TransactionArrival and Expiry

Transactions arrive according to a Poisson process with rate λ . Every transaction has its own gas demand g_n , a max fee f_n^{max} , and a tip π_n . The gas demand and max fee follow a LogNormal distribution, and the tip is determined using an exponential distribution. When a transaction arrives, it is put into the mempool and an expiry event is scheduled, which ensures any unconfirmed transactions leave the mempool 300 seconds after being inserted.

2.2 BlockProduction

On average, a new block is produced every 12 seconds, since the inter-block time is $T_{block} \sim \text{Exp}(1/12)$ seconds. Everytime a new block is produced, an UpdateBaseFee event is triggered and it is decided which transactions to add to the block. The way this is decided is by sorting all transactions in the mempool in order of descending tip size, meaning the first transaction has the highest tip, and the last one has the lowest tip. Then, start scanning at the first transaction and add it to the block if its gas demand fits in the remaining gas budget of the block. Then continue scanning the next one. If the gas demand of a transaction does not fit in the remaining gas budget, the transaction remains in the mempool and the next transaction will be considered.

2.3 UpdateBaseFee

After the production of a new block, the base fee is updated according to the EIP-1559 rule:

$$b_{t+1} = b_t \left(1 + \frac{1}{8} \cdot \frac{G_{used} - G^*}{G^*} \right).$$

Here G_{used} is the total gas consumed by the previous block and G^* is the gas target, which is the same for every block. This way of updating the base fee ensures that the base fee goes up if the previous block was fuller than the target ($G_{used} > G^*$) and goes down otherwise. However, the base fee can not fall below a predetermined minimum base fee $b_{min} = 1$ Gwei, meaning we set $b_{t+1} \leftarrow \max\{b_{t+1}, b_{min}\}$.

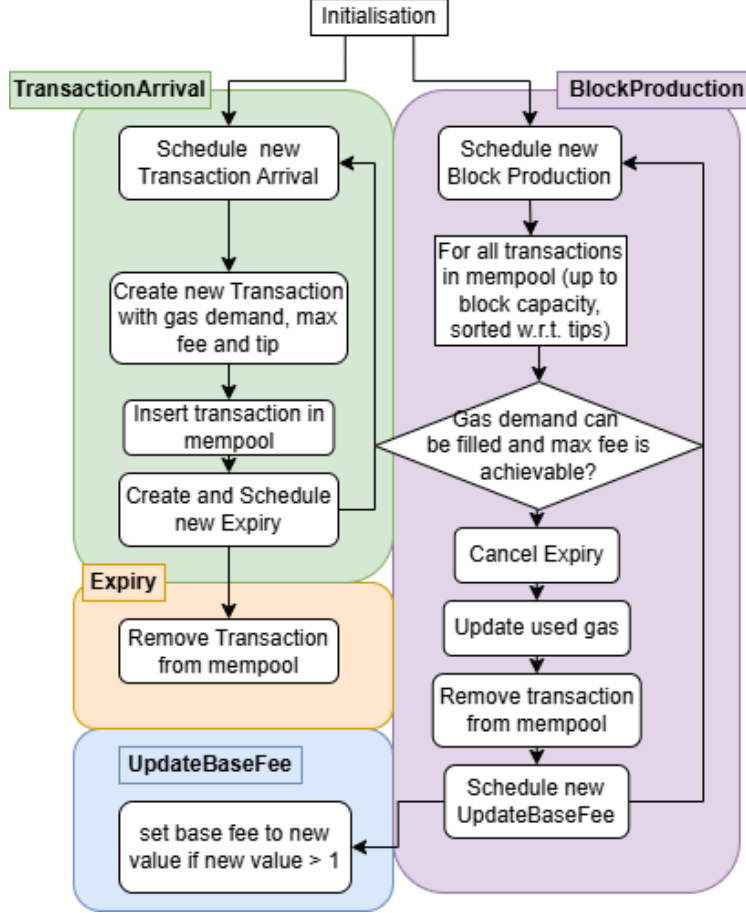


Figure 1: Flowchart Events

3 Batch method

The batch method divides the total runtime into a warm-up period and r batches. Then, in each batch, it averages the statistic(s) of interest, say X , and saves that to a new batch statistic. The new estimator of $\mathbb{E}[X]$ is now given by the mean of the batch statistic.

We implement this by checking in which batch we are every time we update a statistic, and if we move up a batch, we reset all statistics and save their averages to a batch statistic. Then, we report the average of the batch statistics.

3.1 Warm-up Period

To find the required warm up period, we will use the heuristic procedure described in the slides. That is, we will do N_W runs of length K_W . Then, if we let $X_{n,k}$ be the k -th observation of the n -th simulation run of statistic X , and define

$$\bar{X}_{*,k} := \frac{1}{N_W} \sum_{n=1}^{N_W} X_{n,k},$$

the end of the warm-up period, say D , should satisfy

$$\left| \frac{\frac{1}{2D} \sum_{k=1}^{2D} \bar{X}_{*,k}}{\frac{1}{D} \sum_{k=1}^D \bar{X}_{*,k}} - 1 \right| \leq 0.05. \quad (1)$$

We computed this for different warm-up periods, over $N_W = 1$ simulation of length 100,000 time units.

Warm-up Period (time units)	Statistic	Expression in Equation (1)
1	Confirmation time	0.5000
	mempool size	0.5000
	block gas utilisation	0.5001
	base fee	0.5005
50	Confirmation time	0.5001
	Mempool size	0.5004
	Block gas utilisation	0.5002
	base fee	0.5010
100	Confirmation time	0.5009
	Mempool size	0.5012
	Block gas utilisation	0.5006
	Base fee	0.5021
1,000	Confirmation time	0.5048
	Mempool size	0.5048
	Block gas utilisation	0.5038
	Base fee	0.5040
10,000	Confirmation time	0.5562
	Mempool size	0.5560
	Block gas utilisation	0.5586
	Base fee	0.5565

Table 1: Values of the expression in Equation (1) for different warm-up periods.

From these results, it seems that the shorter our warm-up period is, the better. Therefore, we have decided to run our simulation without warm-up period.

3.2 Number of Batches

To find a sufficient number of batches for our simulation, we check the following condition. For a statistic X , number of batches r , batch estimators $\hat{X}, S^2(r)$ for $\mathbb{E}[X]$ and $\text{Var}(X)$ respectively, a precision $\delta > 0$ and

confidence error $\alpha > 0$, we need

$$r \geq \frac{t_{r-1, 1-\frac{\alpha}{2}}^2 S^2(r)}{\frac{\delta}{1+\delta} \hat{X}}, \quad (2)$$

Where $t_{r-1, 1-\frac{\alpha}{2}}$ is the $1 - \frac{\alpha}{2}$ quantile of the student t distribution with $r - 1$ degrees of freedom. We found that, for $\alpha = 0.05, \delta = 0.1$, a number of batches $r = 50$ is sufficient (i.e. Equation 2 holds) for all statistics we keep track of, if we have a simulation length of at least 200,000 time units.

4 Regenerative method

The regenerative method works similarly to the batch method, but instead of fixed times when we reset statistics and save the average, we do so every time some state is reached. The state we use, is when the mempool is empty. We implement this by checking whether the mempool is empty everytime it is changed by either an Expriy or BlockProduction event, and when it is empty, we reset and save our statistics.

5 Verification Cases

5.1 M/M/1 Queue

In the first verification case, we disable transaction expiry, eligibility checks and base fee updates, and we set the block capacity to 1. This reduces the system to an M/M/1 queue. We want to compare the average confirmation time of transactions with the expected waiting time of customers in such a queue. The transaction and block arrival rates will be set as

$$\lambda = 0.05 \quad \mu = \frac{1}{12}.$$

We first compute $\mathbb{E}[W]$, being the expected sojourn time of customers, and $\mathbb{E}[W_q]$ being the average time a customer spends waiting in the queue.

$$\mathbb{E}[W] = \frac{1}{\mu - \lambda} = 30, \quad \mathbb{E}[W_q] = \frac{\lambda}{\mu(\mu - \lambda)} = 18$$

In our system, we expect the average confirmation time to be around the same as $\mathbb{E}[W]$, since

Avg. confirmation time	Confidence interval
31.1058	[28.8485, 33.3631]

Table 2: Results of verification case 1

For this computation, we only used the batch method. This is because the regenerative method needs the system to return to an empty mempool repeatedly. However, in this case, the mempool was never empty.

5.2 M/D/1 Queue

In the second verification case, we reduce the system even more, by making the block arrivals deterministic with rate $1/12$. We again compute $\mathbb{E}[W]$ and $E[W_q]$.

$$\mathbb{E}[W] = \frac{1}{2\mu(1 - \rho)} = 15 \quad \mathbb{E}[W_q] = \frac{\rho}{2\mu(1 - \rho)} = 9$$

where $\rho = \frac{\lambda}{\mu}$.

Again, the regenerative method fails here, since the mempool is never empty.

Avg. Confirmation time	Confidence interval
14.6274	[13.7656, 15.4892]

Table 3: Results of verification case 2

6 Results

We are asked to compare two models with different parameters $\lambda = 8, \lambda = 12$.

$\lambda = 12$		Batch method		Regenerative method	Full series
Statistic	Avg.	95% CI	Avg.	95% CI	Avg.
Confirmation time	12.1959	[11.9004, 12.4914]	5.9826	[5.8921, 6.0731]	1.0239
Mempool size	146.4415	[142.8648, 150.0183]	71.8253	[70.7335, 72.9171]	
block gas util.	0.2387	[0.2351, 0.2423]	0.2304	[0.2272, 0.2337]	
base fee	1.0197	[1.000, 1.0393]	1.0181	[1.0149, 1.0213]	
expiry rate	0.0000	[0.0000, 0.0000]	0.0000	[0.0000, 0.0000]	
ineligible fraction	0.0050	[0.0029, 0.0071]	0.0055	[0.0044, 0.0067]	
$\lambda = 8$		Batch method		Regenerative method	Full series
Statistic	Avg.	95% CI	Avg.	95% CI	Avg.
Confirmation time	11.9496	[11.7236, 12.1756]	6.0660	[5.9752, 6.1568]	1.0106
Mempool size	95.6138	[93.7847, 97.4429]	48.5387	[47.8074, 49.2701]	
block gas util.	0.1598	[0.1573, 0.1623]	0.1591	[0.1568, 0.1615]	
base fee	1.0064	[0.9916, 1.0213]	1.0073	[1.0047, 1.0098]	
expiry rate	0.0000	[0.0000, 0.0000]	0.0000	[0.0000, 0.0000]	
ineligible fraction	0.0103	[0.0087, 0.01204]	0.0101	[0.0086, 0.0116]	

Table 4: Results of comparison test.

6.1 Difference in methods

We see that there is a large difference between batch and regenerative method in Confirmation time and Mempool size. This is likely because these are computed close to when new regenerative cycles are started, and that there may be a lot of smaller cycles. These smaller cycles take the averages of these statistics down drastically, but don't really touch the other statistics, as they don't even have enough time to change. This is supported by the fact that in the end we have 16362 cycles ($\lambda = 12$) and 16444 cycles ($\lambda = 8$).

6.2 Difference in rates

6.2.1 Confirmation time

Between $\lambda = 12$ and $\lambda = 8$, we can see that the average confirmation time is not very different. This probably means when the system is running at a higher rate $\lambda = 12$, it is still able to process transactions at roughly the same speed. It seems that the transaction arrival rate is not a bottleneck for this statistic.

6.2.2 Mempool size

The mempool size, however, does change a lot. The mempool size is way larger with a higher rate, which is to be expected, since the transaction processing speed (block arrival rate) is unchanged.

6.2.3 Block gas utilisation

The block gas utilisation also increases by a noticeable amount. This is also logical, since throughput of transactions has increased, while the waiting time is roughly the same.

6.2.4 Base fee

The base fee also has a noticeable change. With a higher transaction arrival rate, the base fee is also higher. This is, again, expected, since more gas is being used. Because the change in base fee is proportional to the difference of used gas with the gas target, if we have a higher gas utilisation, we will a base fee that at least decreases slower.

6.2.5 Expiry rate

In all cases, the expiry rate is 0. This means that the system is still able to handle all transactions. We would need to either increase the transaction arrival rate or decrease the block arrival rate to see any expiries. This is logical, since the time until expiry is fixed at 300 seconds. In this time, on average $300\mu = 25$ blocks will have had the chance to process this transaction before it expires.

6.2.6 Fraction of ineligible transactions

The fraction of ineligible transactions is higher for $\lambda = 8$. This can be explained by the fact that the average mempool size is way smaller, since the statistic is computed as

$$\frac{\# \text{ ineligible transactions in mempool}}{\# \text{ transactions in mempool}}.$$

It seems this has a larger effect than the base fee increase.